

Network Scanner Scripts

Pulling, running, and configuring `network_scanner.py` and `mobile_network_scanner.py`

Repository: <https://github.com/thomasmd321/Test-repo>

Branch: `claude/local-network-device-discovery-joawq5`

Both scripts discover devices on your local network with no external service and no account required. **network_scanner.py** targets a normal desktop/Termux environment and can read ARP/MAC data directly; **mobile_network_scanner.py** targets a sandboxed environment such as an iPhone running a-Shell, where the OS blocks raw sockets and subprocess calls, so it relies on plain TCP connections instead.

	<code>network_scanner.py</code>	<code>mobile_network_scanner.py</code>
Target platform	Linux / macOS / Windows / Termux	iOS sandboxes (a-Shell, Pythonista)
Discovery method	ARP scan (scapy), ping-sweep fallback	Plain TCP connect probe
MAC address / vendor	Yes (IEEE OUI lookup)	No (not exposed to the sandbox)
IPv6 discovery	Yes (<code>--ipv6</code>)	No
Banner grabbing	<code>--identify</code> IP deep dive	Built into every scan
Risky-port flagging	Yes	Yes
NEW / CHG / missing tracking	Yes	Yes
IP-conflict / spoofing alert	Yes	No (no MAC available)
Export results	<code>--output</code> FILE (CSV/JSON)	<code>--output</code> FILE (CSV/JSON)
Scan history log	<code>--log-history</code> FILE	<code>--log-history</code> FILE
Excluding hosts	<code>--exclude</code> IP/CIDR	<code>--exclude</code> IP/CIDR
Custom device labels	<code>--set-label</code> / <code>--remove-label</code>	<code>--set-label</code> / <code>--remove-label</code>

Environment diagnostics	--doctor	--doctor
Quiet mode (cron/systemd)	--quiet	--quiet
Webhook notifications	--notify-webhook	--notify-webhook
Multi-subnet scanning	--all-subnets (parallel)	Manual comma-separated list
Shell tab-completion	completions.bash	completions.bash

1. Getting the scripts

Both scripts are plain, dependency-light Python files living in one repository. Pick whichever pull method fits where you're running them.

Clone the whole repository (desktop / Termux)

```
git clone https://github.com/thomasmd321/Test-repo.git
cd Test-repo
git checkout claude/local-network-device-discovery-joawq5
```

Grab a single file directly (no git needed)

Useful on iOS, or anywhere you only want one script without the rest of the repo:

```
BASE=https://raw.githubusercontent.com/thomasmd321/Test-repo/claude/local-network-device-discovery-joawq5
curl -O $BASE/network_scanner.py
curl -O $BASE/mobile_network_scanner.py
```

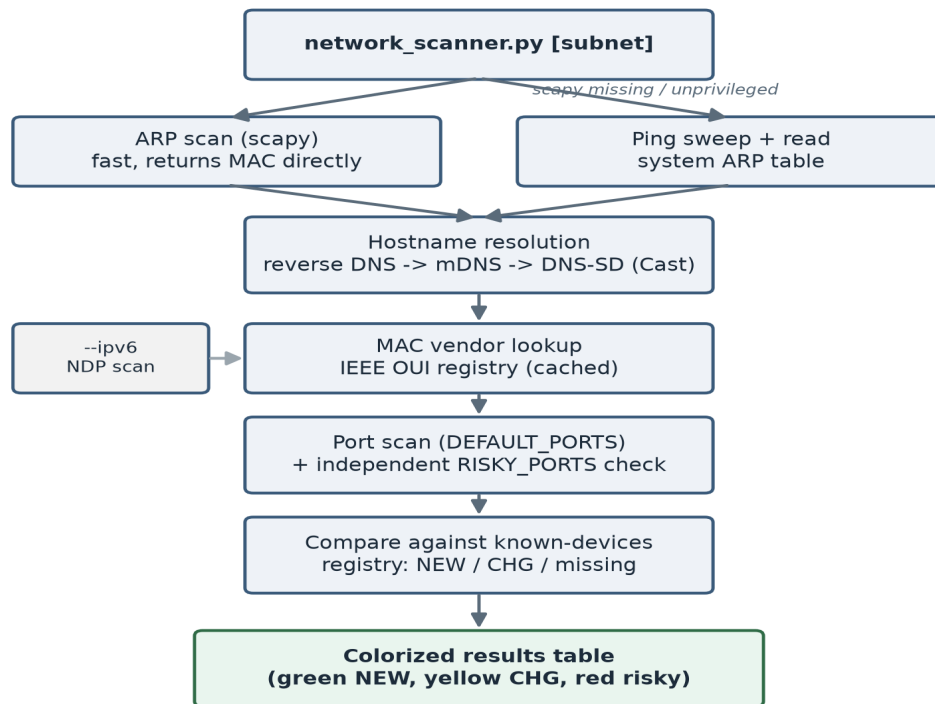
`curl -O` saves the file under its own name in the current directory and overwrites an existing copy, so re-running the same command later pulls in any updates.

On an iPhone

Install **a-Shell** from the App Store (not “a-Shell mini,” which strips out `git`), then run either the clone or the single-file `curl` command above from its terminal.

2. network_scanner.py

For a normal desktop, laptop, or Termux-on-Android environment. Prefers a fast ARP scan via scapy (returns MAC addresses directly); if scapy isn't installed or the process lacks the privileges ARP scanning needs, it falls back automatically to a multithreaded ping sweep plus the system's own ARP table — no flag needed to trigger this, it just happens.



Falls back automatically when scapy / raw sockets aren't available.

How a scan flows through network_scanner.py, start to finish.

Running it

```
python3 network_scanner.py                # auto-detect subnet
python3 network_scanner.py 192.168.1.0/24  # scan a specific subnet
python3 network_scanner.py --all-subnets   # every subnet (needs psutil)
python3 network_scanner.py --identify 192.168.1.42 # deep-dive one host
python3 network_scanner.py --watch 300      # rescan every 5 minutes
python3 network_scanner.py --output scan.json # save results to a file
```

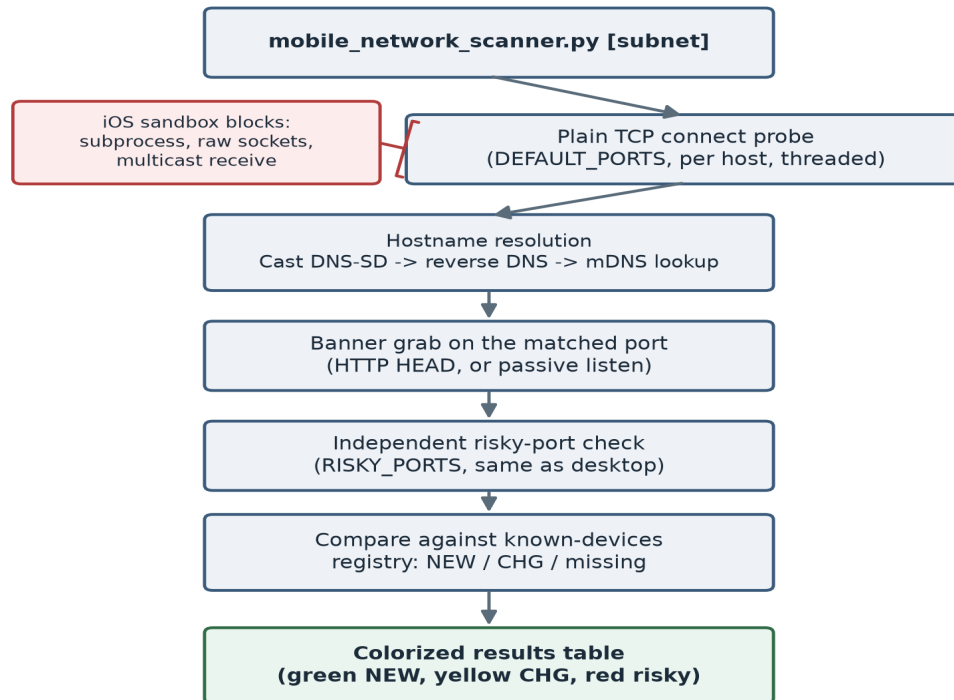
Options

Flag	What it does
subnet	Positional. CIDR range(s) to scan, comma-separated for more than one. Auto-detected if omitted.
--all-subnets	Scan every local subnet this machine has an interface on (requires <code>pip install psutil</code>) instead of just the default route. Subnets are scanned concurrently, not one at a time, so several VLANs don't multiply the wall-clock cost.

Flag	What it does
--identify IP	Skip the network scan; do a slow, thorough single-host investigation instead (more ports, banner grabs, full hostname/vendor resolution).
--doctor	Skip the network scan; check this environment for everything this script can use (scapy, ping/arp, cache writability, mDNS, etc.) and report it.
--timeout SECONDS	Timeout per host (default: 1.0).
--mdns-timeout SECONDS	Timeout for the mDNS/DNS-SD hostname fallback (default: 0.3).
--no-vendor-lookup	Skip the IEEE OUI vendor lookup (avoids the first-run registry download).
--refresh-vendor-db	Force a fresh OUI registry download instead of using the cached copy.
--watch SECONDS	Rescan on a timer instead of once, printing NEW markers as they appear (Ctrl+C to stop).
--no-track-devices	Don't use the known-devices registry at all — no NEW/CHG markers, nothing remembered.
--forget-known-devices	Clear the registry first, so everything in this run shows as NEW.
--set-label KEY=LABEL	Assign a friendly label to a device (KEY is its MAC, or IP if it has none), shown instead of/alongside its hostname. Repeatable.
--remove-label KEY	Remove a device's custom label. Repeatable.
--ipv6	Also discover IPv6 devices via multicast ping + NDP (Linux/macOS only).
--ipv6-timeout SECONDS	Roughly how long to spend on IPv6 discovery (default: 2.0).
--no-scan-ports	Skip the open-port probe (and the risky-ports check that depends on it).
--ports LIST	Comma-separated TCP ports to probe instead of the built-in default list.
--exclude LIST	Comma-separated IPs and/or CIDR ranges to drop from the scan right after discovery (a bare IP is treated as a /32).
--port-timeout SECONDS	Per-port connection timeout, for both the port probe and risky-ports check (default: 0.3).
--no-risky-ports	Skip the risky-ports security check while keeping the general port probe.
--no-color	Disable ANSI color output (also respects the NO_COLOR env var).
--output FILE	Save this scan's results to FILE as JSON, or CSV if it ends in .csv.
--log-history FILE	Append this scan's results as one JSON line to FILE, instead of overwriting it like --output does.
--history-max-entries N	Cap FILE from --log-history at N lines, dropping the oldest first (default: 200).
--quiet	Print nothing for a scan with no NEW/CHG/missing/risky devices - only an interesting run produces output (see --watch under cron/systemd).
--notify-webhook URL	POST a summary to URL as Slack-compatible JSON whenever a scan has something to report - same trigger as --quiet.

3. mobile_network_scanner.py

For sandboxed Python runtimes — most notably iOS apps like a-Shell — that allow neither subprocess calls nor raw sockets. Every discovery technique here is built from ordinary client TCP/UDP sockets, which is the one thing every one of these sandboxes still permits.



No ARP, no IPv6 discovery, no MAC vendor lookup - ordinary sockets only.

How a scan flows through mobile_network_scanner.py, start to finish.

Running it

```
python3 mobile_network_scanner.py           # auto-detect subnet
python3 mobile_network_scanner.py 192.168.1.0/24 # scan a specific subnet
python3 mobile_network_scanner.py --timeout 0.5 --ports 22,80,443
python3 mobile_network_scanner.py --watch 300      # rescan every 5 minutes
python3 mobile_network_scanner.py --no-banners --no-risky-ports # faster scan
python3 mobile_network_scanner.py --output scan.csv # save results to a file
```

Options

Flag	What it does
subnet	Positional. CIDR range(s) to scan, comma-separated for more than one. Auto-detected if omitted.
--doctor	Skip the network scan; check this environment for everything this script can use (subnet detection, TCP connectivity, cache writability, mDNS) and report it.
--timeout SECONDS	Timeout per port probe (default: 0.5); also used for the banner-grab and risky-ports steps.

Flag	What it does
--ports LIST	Comma-separated TCP ports to probe instead of the built-in default list.
--exclude LIST	Comma-separated IPs and/or CIDR ranges to drop before probing anything (a bare IP is treated as a /32).
--mdns-timeout SECONDS	Timeout for the mDNS/Bonjour hostname fallback (default: 0.3).
--watch SECONDS	Rescan on a timer instead of once, printing NEW markers as they appear (Ctrl+C to stop).
--no-track-devices	Don't use the known-devices registry at all — no NEW/CHG markers, nothing remembered.
--forget-known-devices	Clear the registry first, so everything in this run shows as NEW.
--set-label KEY=LABEL	Assign a friendly label to a device (KEY is its IP address), shown instead of/alongside its hostname. Repeatable.
--remove-label KEY	Remove a device's custom label. Repeatable.
--no-banners	Skip banner grabbing on each device's open port — faster, but a weaker identification hint.
--no-risky-ports	Skip the risky-ports security check while keeping the general port probe.
--no-color	Disable ANSI color output (also respects the NO_COLOR env var).
--output FILE	Save this scan's results to FILE as JSON, or CSV if it ends in .csv.
--log-history FILE	Append this scan's results as one JSON line to FILE, instead of overwriting it like --output does.
--history-max-entries N	Cap FILE from --log-history at N lines, dropping the oldest first (default: 200).
--quiet	Print nothing for a scan with no NEW/CHG/missing/risky devices - only an interesting run produces output (see --watch under cron/systemd).
--notify-webhook URL	POST a summary to URL as Slack-compatible JSON whenever a scan has something to report - same trigger as --quiet.

iOS limitation: mDNS/DNS-SD queries fail outright on iOS with `0SError(65, 'No route to host')` — Apple's Local Network Privacy model requires an app bundle to declare Bonjour service types in advance, which a script run from a generic terminal app can't do. Plain TCP port scanning, banner grabbing, and risky-port checks are unaffected, since they're ordinary unicast traffic.

4. Known-device tracking, color coding, and --watch

Both scripts persist a small local registry of every device they've ever seen

(`~/.cache/network_scanner_known_devices.json` and

`~/.cache/mobile_network_scanner_known_devices.json`). Every scan is compared against it, producing the markers and colors below. Pair either script with `--watch SECONDS` to turn a one-shot scan into a standing monitor.

```
python3 network_scanner.py

Scanning 192.168.1.0/24 ...

  IP Address      MAC Address      Vendor      Port  Service  Hostname
-----
NEW 192.168.1.42    3c:22:fb:1a:9c:04 Apple, Inc  62078 lockdownd iPhone.local
192.168.1.1      aa:bb:cc:dd:ee:ff Netgear    80    http    router.local
CHG 192.168.1.57    10:2c:6b:44:2a:11 Sonos      23    ?       kitchen-speaker.local
192.168.1.72     b0:c5:54:2f:88:19 Google     8009  chromecast Living Room TV
192.168.1.88     f4:5e:ab:12:34:56 Synology   445    smb     nas.local      (risky)

5 device(s) found. 1 new since last seen.
2 previously-seen device(s) not found in this scan:
192.168.1.63      last seen 2026-09-14T08:03:00 (laptop.local)
1 device(s) with a changed port since last seen:
192.168.1.57      (none) -> telnet (23)
1 device(s) exposing commonly-risky ports:
192.168.1.88     f4:5e:ab:12:34:56  smb (445)
```

Illustrative output — exact columns vary slightly between the two scripts.

Marker / color	Meaning
NEW (green)	First time this device has ever been seen.
CHG (yellow)	A known device is answering on a different port than last time.
red row	Device exposes a port on the risky-ports list (telnet, SMB, RDP, VNC, FTP).
magenta row	This IP was last attributed to a different MAC address (network_scanner.py only).
dim gray line	A previously-seen device that didn't show up in this scan.

A device can be NEW or CHG, never both — a device with no prior registry entry can't have a “changed” port, only a first one. A device can be NEW *and* a magenta IP conflict at once, though (a freshly-added device can land on an IP someone else just gave up) — row-color priority, most to least urgent: risky, conflict, new, changed port.

Resetting tracking

```
python3 network_scanner.py --no-track-devices      # skip tracking this run
python3 network_scanner.py --forget-known-devices  # reset registry, all NEW
python3 mobile_network_scanner.py --watch 300      # standing monitor on phone
```

Custom device labels

The same registry can hold a friendly label for a device — useful when its real hostname is cryptic or blank — shown in place of/alongside the hostname (e.g. Kitchen Server (localhost) when they differ). KEY is a device's MAC (or IP if it has none) for `network_scanner.py`, or always its IP for `mobile_network_scanner.py`.

```
python3 network_scanner.py --set-label aa:bb:cc:dd:ee:ff="Kitchen Server"
python3 mobile_network_scanner.py --set-label 192.168.1.42="Kitchen Echo"
python3 network_scanner.py --remove-label aa:bb:cc:dd:ee:ff
```

Both flags are repeatable and take effect immediately, including on the scan that runs in the same command. A device doesn't need to already be in the registry — labeling one creates a minimal entry for it, though that also means it won't show as NEW the next time it's actually scanned.

IP-conflict / spoofing alerts (`network_scanner.py` only)

The same registry also catches a device's IP being taken over by a different MAC address than last time — a DHCP lease getting handed to a new device is the ordinary cause, but it's exactly the same signal something spoofing another device's IP (most notably ARP-poisoning your router's own address) would produce. A conflicting device prints in magenta and shows up in its own summary section.

This is a hygiene signal, not an intrusion-detection system — most hits will be completely benign lease reassignments. A conflict on your router/gateway's own IP is the one case worth treating as urgent rather than routine. Only compares devices that both have a real MAC address, so a ping-sweep-only device (no MAC at all) can't trigger or be flagged by this. Not available on `mobile_network_scanner.py`, which has no MAC address to compare in the first place. Skipped entirely with `--no-track-devices`.

5. Exporting results

Both scripts can save a scan's results to a file with `--output FILE`, independent of the known-devices registry above — useful for feeding results into another tool, diffing two scans, or just keeping a dated record. The format is chosen by the file extension: `.csv` writes a CSV file, anything else (typically `.json`) writes JSON.

```
python3 network_scanner.py --output scan.json
python3 network_scanner.py --output scan.csv
python3 mobile_network_scanner.py --output scan.json
```

Each run overwrites FILE with that scan's results — this is a snapshot of the latest scan, not an appended history log.

6. Diffing two scans

scan_diff.py is a third, standalone script that compares two files saved with --output above and reports what changed: devices added, devices that disappeared, and per-field changes (a different port, hostname, banner, etc.) on devices present in both — the same NEW/missing/CHG comparison the scanners do against their own known-devices registry, just applied to two snapshots on disk instead.

```
python3 scan_diff.py old.json new.json
python3 scan_diff.py before.csv after.csv
python3 scan_diff.py --no-color old.json new.json
```

Either file can be JSON or CSV independently (comparing a CSV export against a JSON one works fine), and they don't need to come from the same script: fields the two scanners don't share (mac/vendor are desktop-only, banner is mobile-only) are still compared whenever both files happen to have them. A device is matched across the two files by MAC when present, falling back to IP — the same identity rule network_scanner.py's own known-devices tracking uses.

7. Quiet mode and environment diagnostics

--quiet

Suppresses everything — even the scan's own “Scanning...” line — for a run with nothing to report: no NEW devices, no port changes, no missing devices, and no risky ports. As soon as one of those is true, the full report prints exactly as it would without --quiet. Meant for --watch under cron/systemd, where a boring rescan producing zero output (rather than a full table every time) is what makes “did anything happen” easy to grep for or alert on.

```
python3 network_scanner.py --watch 300 --quiet
python3 mobile_network_scanner.py --watch 300 --quiet
```

--doctor

Skips the network scan and instead checks this environment for everything the script can use — scapy, ping/arp, cache-directory writability, mDNS multicast, and so on — reporting each with a clear pass/fail and, for most, the fallback that kicks in when it fails. Exits 0 if every check passed, 1 otherwise, so it's usable as a pre-flight check in a script.

```
python3 network_scanner.py --doctor
python3 mobile_network_scanner.py --doctor
```

A failed check doesn't necessarily mean a scan will fail — most have a documented fallback (no arp on PATH just means no MAC from that path, not a crash). The one exception: a failed mDNS check on `mobile_network_scanner.py` almost always means iOS's Local Network Privacy restriction (see `mdns_diagnostic.py`), which no retry or code change here can fix.

8. Notifications for --watch

--notify-webhook URL POSTs a plain-text summary to URL as {"text": "..."} JSON — the format Slack's incoming webhooks (and many other generic webhook receivers) expect directly — whenever a scan has a NEW device, a port change, a missing device, or a risky port to report. A boring scan sends nothing at all, the same trigger condition --quiet uses, so the two pair naturally under --watch.

```
python3 network_scanner.py --watch 300 \  
  --notify-webhook https://hooks.slack.com/services/...  
python3 mobile_network_scanner.py --watch 300 \  
  --notify-webhook https://ntfy.sh/your-topic
```

A failed or unreachable webhook prints a warning to stderr and the scan continues normally — it never crashes the run. This deliberately doesn't special-case any one service's exact payload shape; a target expecting something else (Discord's content key, ntfy.sh's plain-text body) may need a small relay in between, or just point it at a service that already speaks the Slack-compatible format.

9. Excluding devices from a scan

--exclude IP/CIDR skips one or more addresses — comma-separated, each either a bare IP (treated as a /32) or a CIDR range — from the rest of a scan, without narrowing the whole subnet just to dodge one host: a printer that crashes under port probes, a NAS you don't want woken from sleep, or a noisy neighbor you just don't care about.

```
python3 network_scanner.py --exclude 192.168.1.5
python3 network_scanner.py --exclude 192.168.1.5,192.168.1.10
python3 network_scanner.py --exclude 192.168.1.0/28
python3 mobile_network_scanner.py --exclude 192.168.1.5,10.0.0.0/24
```

On `mobile_network_scanner.py`, exclusion is complete: excluded hosts are dropped from the list before any TCP connection is ever attempted, so nothing at all reaches them. On `network_scanner.py`'s ARP path, the initial ARP broadcast still reaches every host on the subnet — `scapy`'s `srp()` sends one request across the whole range in a single call, with no way to carve individual addresses out of that broadcast — but excluded devices are dropped immediately after discovery, before vendor lookup, port scanning, the risky-ports check, and the final report/export/tracking, so in practice they're never touched beyond that one broadcast packet. The ping-sweep fallback still pings every host itself and filters after, since it has no equivalent single-broadcast step to route around.

10. Scan history log

`--log-history FILE` appends every scan's results to FILE as one JSON line per run (`{"timestamp": "...", "devices": [...]}`), instead of overwriting it like `--output` does — useful under `--watch` for keeping a record of what the network looked like over time, or for feeding into your own analysis later (each line parses independently, so you don't need to load the whole file to read one entry).

```
python3 network_scanner.py --watch 300 --log-history history.jsonl
python3 mobile_network_scanner.py --log-history history.jsonl
```

The file is capped at 200 entries by default (oldest dropped first) so it doesn't grow forever under a long-running `--watch`; override with `--history-max-entries N`.

```
python3 network_scanner.py --watch 300 --log-history history.jsonl \
--history-max-entries 1000
```

11. Shell tab-completion

completions.bash adds bash tab-completion for each script's flag names (18+ per script by now, easy to half-remember). Source it from your ~/.bashrc:

```
source /path/to/Test-repo/completions.bash
```

It only completes flag *names*, not their arguments (a subnet, a port list, a file path), and only fires for a direct invocation matching a script's own name (./network_scanner.py, or the bare name if it's on PATH — the scripts in this repo already have their executable bit set). python3

network_scanner.py <TAB> does **not** trigger it: bash keys completion off the first word of the command line, which is python3 in that case, not the script — putting the script on PATH so the bare-name form works is the practical fix.